23L-0588 — VisionGuard

AI-generated versus real-image classification service

Student: Muhammad Abdul Raheem

Project: 23L-0588_A2_Ops

GitHub: github.com/abdul-raheem-fast/23L-0588_A2_Ops

DagsHub: dagshub.com/abdul-raheem-fast/23L-0588_A2_Ops

Live URL: 23l-0588-a2-ops.ghauri.prod.page

Final repository commit: d03b753

This report documents the data audit, model development, experiment tracking, Flask service, database trail, containerization, deployment, and retained execution evidence for VisionGuard.

1. Data understanding and versioning

1.1 Environment

The workflow was executed in an isolated Colab runtime. A T4 GPU was selected and TensorFlow GPU visibility was checked before training. TensorFlow 2.21.0 was used because TensorFlow 2.18.0 was unavailable in the runtime; the actual version is recorded rather than hidden.

1.2 Dataset audit

The audit script records class label, file size, dimensions, colour mode, file format, SHA-256 content hash, readability, and duplicate status for every image. The run audited 975 images: 539 from AiArtData and 436 from RealArt. No files were unreadable, and two exact duplicates were identified by matching content hashes. Stratified splits were used to preserve class proportions.

1.3 Visual EDA

The EDA describes the source collection before training. The sample grid shows representative examples from both classes, while the width, height, and file-size distributions show substantial variation, including very large source images. This supports fixed-size preprocessing and batched loading, but the distributions are not treated as proof of a class-specific visual feature.

1.4 DVC

The repository includes dvc.yaml and params.yaml. The pipeline separates dataset auditing from split generation and records the seed and image size in configuration. A separate authenticated DVC push/pull screenshot was not captured, so remote transfer is not claimed as completed evidence.

2. Model development

Input pipeline

The tf.data pipeline reads reproducible split CSVs, decodes images, converts them to RGB, resizes them to 224×224, batches samples, and prefetches batches. Training-only augmentation is applied before batching; validation and test images are resized and normalized without random augmentation. The square target provides a consistent tensor shape but can change the original aspect ratio.

Model comparison

The baseline CNN uses convolution, pooling, normalization, dropout, and a sigmoid output for binary classification. MobileNetV2 was loaded without its classifier, trained first with the backbone frozen, and then fine-tuned by unfreezing the final block at a lower learning rate. Early stopping, checkpointing on validation loss, and learning-rate reduction were used to retain the best generalizing checkpoint.

Evaluation

The fine-tuned MobileNetV2 model was selected as champion. Test metrics were accuracy 0.7449, precision 0.8718, recall 0.6296, F1 0.7312, and ROC-AUC 0.8695. The precision-oriented operating point reduces the risk of incorrectly calling a real photograph AI-generated, which is the more damaging user-facing error for this product.

Error analysis and tuning

The evidence notebook generated a confusion matrix, ROC curve, error-analysis CSV and ten-image error grid. It also ran eight Keras Tuner trials covering the requested tuning workflow. The retained error grid is shown in the evidence appendix without altering the submitted images.

3. MLflow and DagsHub

Tracking

MLflow tracking was configured against DagsHub. The executed experiments recorded model family, training configuration, validation history, and final evaluation metrics, allowing the baseline and MobileNetV2 approaches to be compared from their run pages.

Registry evidence

The retained screenshots show DagsHub tracking configuration and completed VisionGuard runs. Independent model-registry version and alias evidence was not captured and is not represented as completed evidence.

4. Flask application and database trail

Application

The model is loaded once during application startup. Upload validation checks extension, size, and decoded image content without persisting the upload. The result displays the predicted label, confidence, and a confidence caveat. The service exposes /health, /predict, and /history.

Persistence

Each successful prediction stores a UTC timestamp, sanitized filename, decision, confidence, model name, and model version. Uploaded image contents are not persisted. SQLite is used by the free deployment, while PostgreSQL is supported through environment-based configuration.

5. Docker and Compose

Container design

The Docker image installs serving dependencies separately from training and MLflow tools. It uses a slim CPU base, runs as a non-root user, downloads the public champion model when it is absent from the build context, and exposes port 8000.

Compose design

Docker Compose defines web and database services, health checks, readiness dependency, and named volumes. The first Blitz build failed because TensorFlow and MLflow imposed incompatible protobuf requirements. Separating the dependency manifests fixed the build without weakening the serving image.

6. Deployment and API verification

Deployment

The application was deployed from the GitHub main branch on Blitz Cloud using the repository Dockerfile. The free service may sleep after inactivity, and no paid cloud credits were used.

Verification

The public health endpoint returned model visionguard-champion, version 1, and status ok. A real dataset image sent to /predict returned an AI-generated decision with 71.3% confidence. Successful requests are recorded for /history.

7. Submission notes

Links

GitHub: github.com/abdul-raheem-fast/23L-0588_A2_Ops
DagsHub: dagshub.com/abdul-raheem-fast/23L-0588_A2_Ops
Live service: 23l-0588-a2-ops.ghauri.prod.page

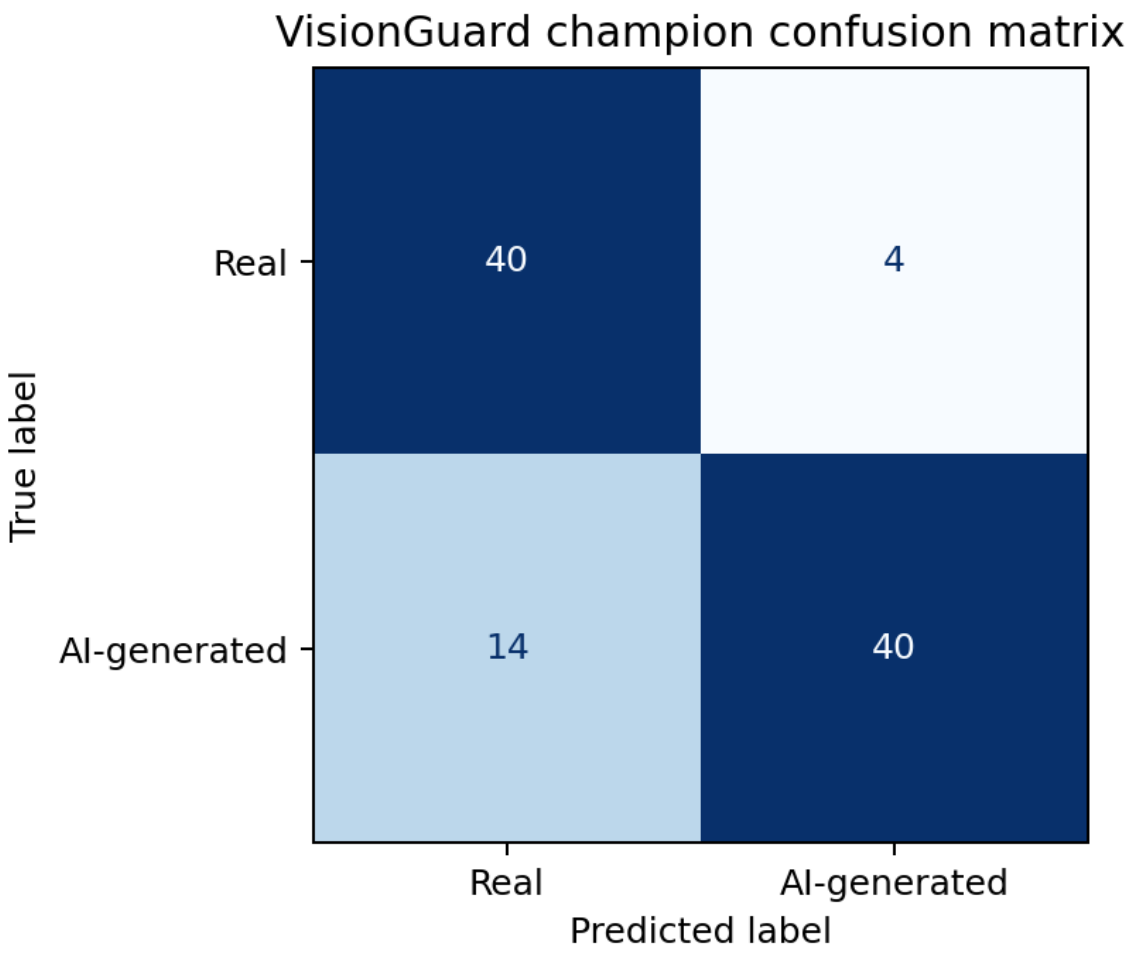
Evidence policy

All retained screenshots in this report are real files from reports/screenshots. No credentials, tokens, raw dataset, local database, or temporary archive is included. Evidence not captured during the run is explicitly identified rather than recreated.

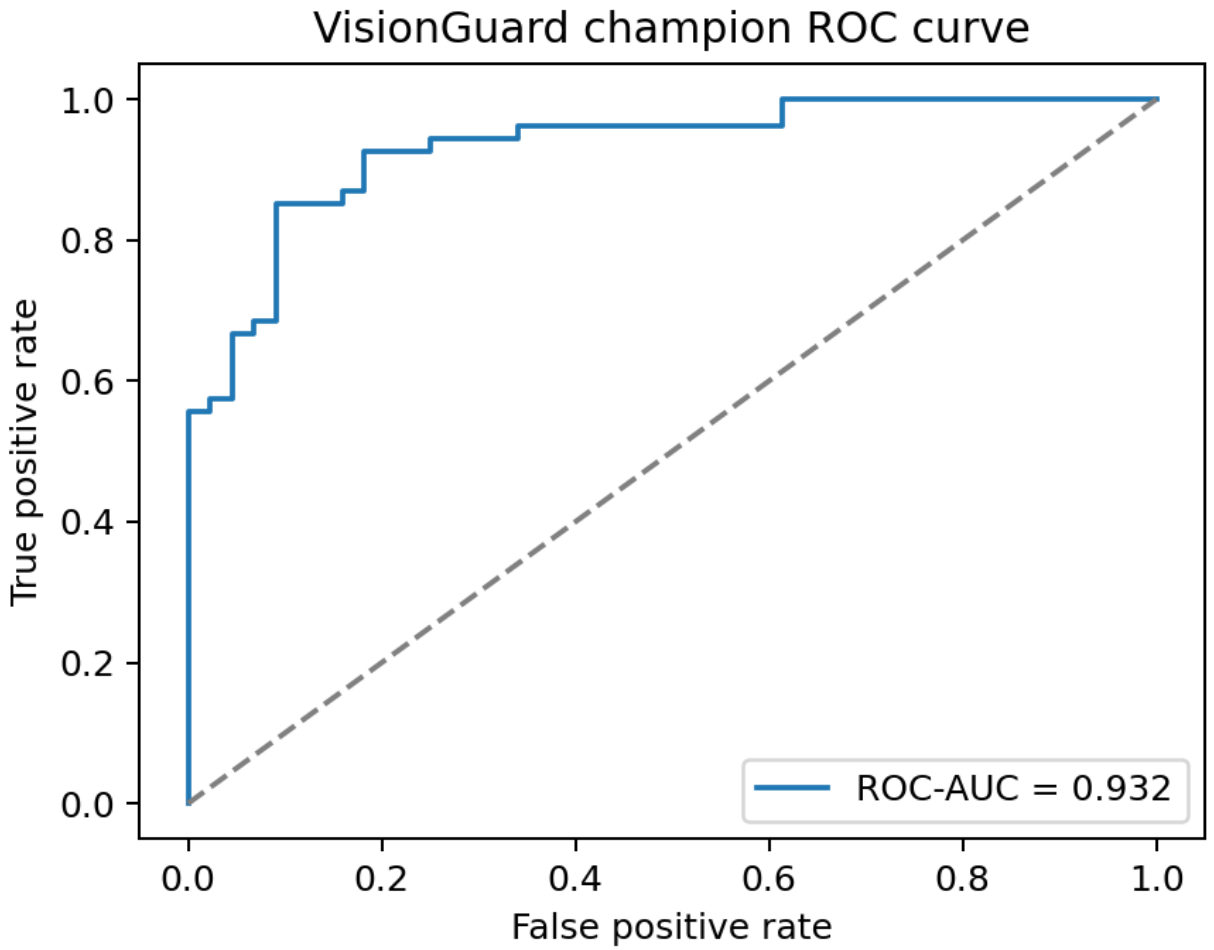
Champion metrics

Metric	Value
Accuracy	0.7449
Precision	0.8718
Recall	0.6296
F1 score	0.7312
ROC-AUC	0.8695

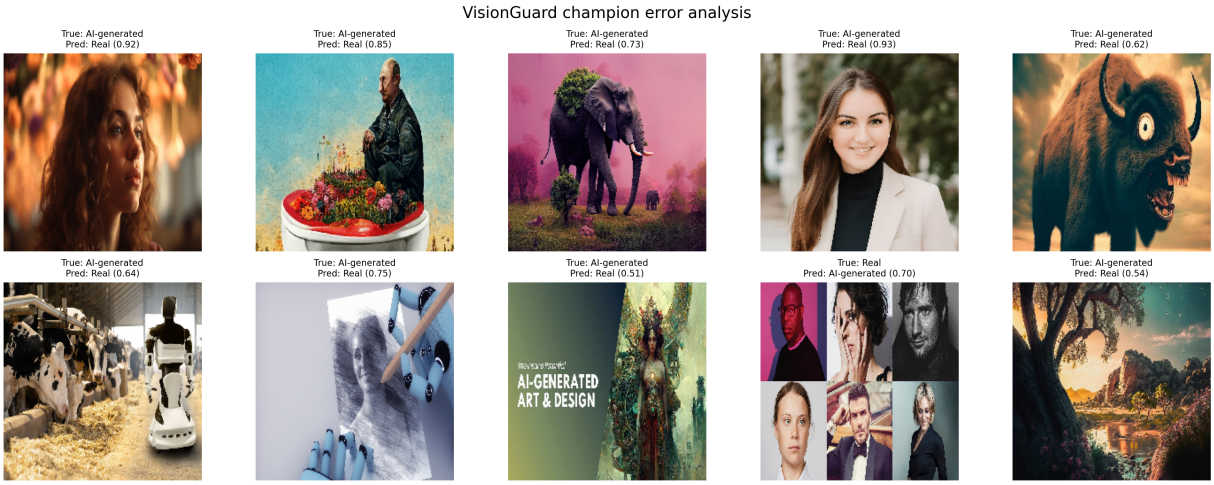
8. Evaluation figures



confusion_matrix.png *— Champion confusion matrix from the Colab evidence run.*

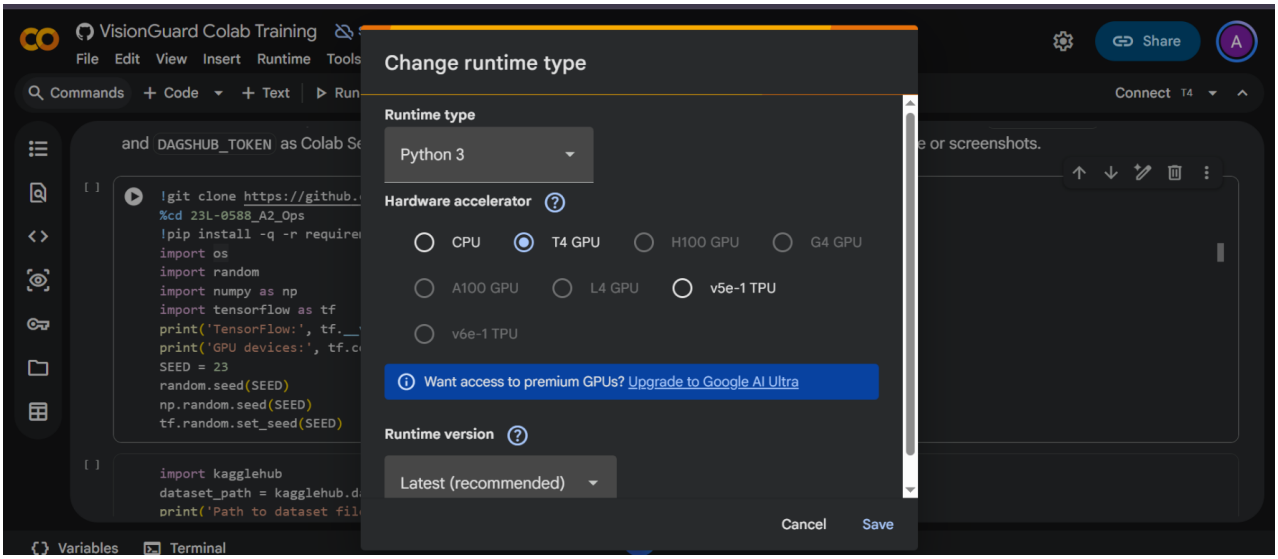


roc_curve.png *— Champion ROC curve and area-under-curve result.*

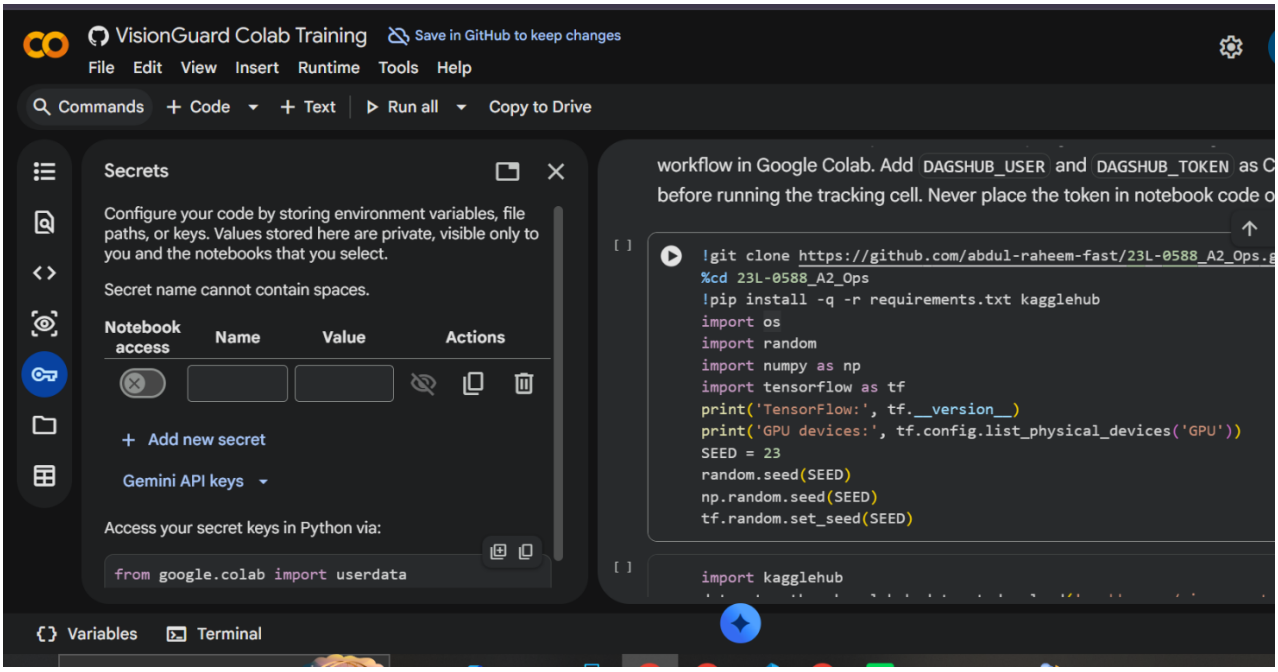


error_analysis_grid.png *— Ten selected test-set errors used for qualitative analysis.*

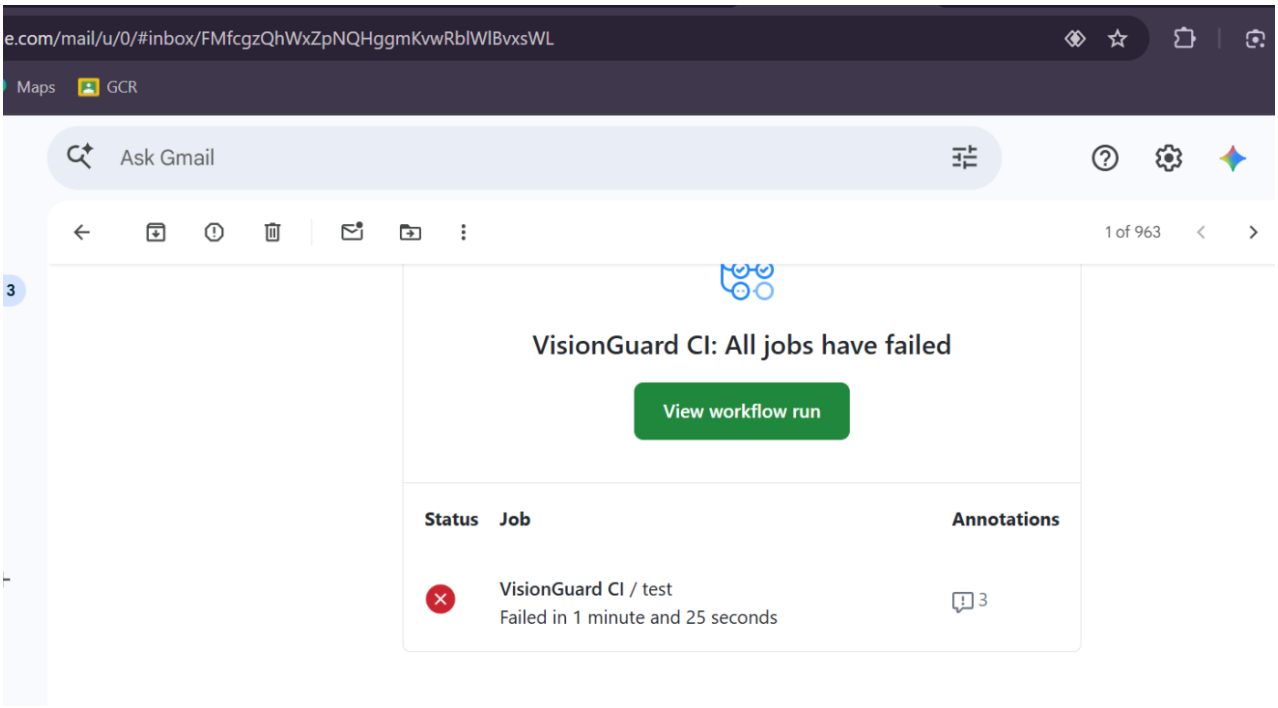
Appendix A — Saved evidence screenshots



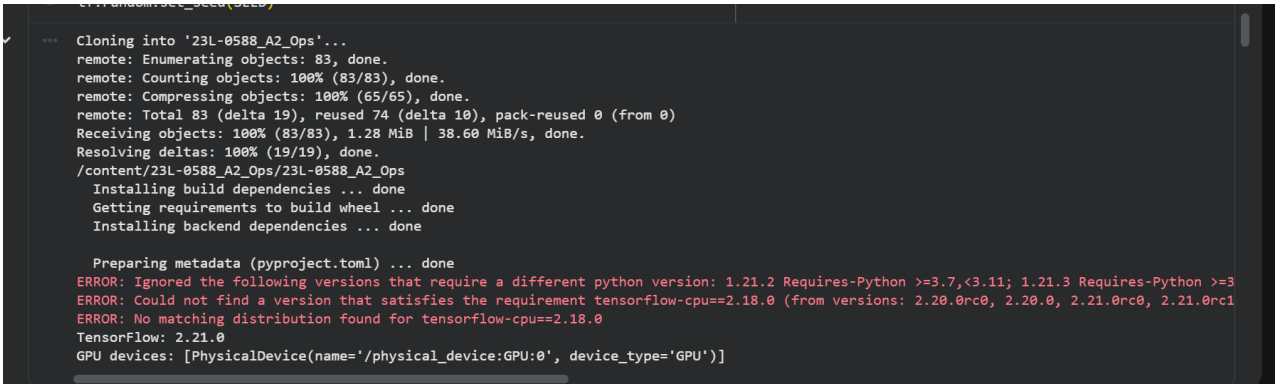
01_colab_runtime_t4.png *— Task 1.1 — Colab runtime setup. This view records the selected T4 GPU hardware used for the training workflow.*



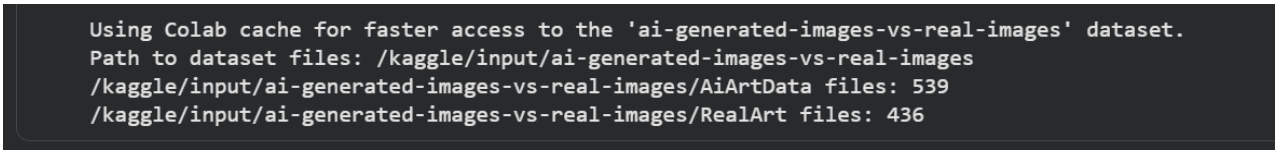
02_colab_secrets_panel.png *— Task 3.2 — Secret handling. The DagsHub credentials are supplied through Colab Secrets rather than written into notebook code.*



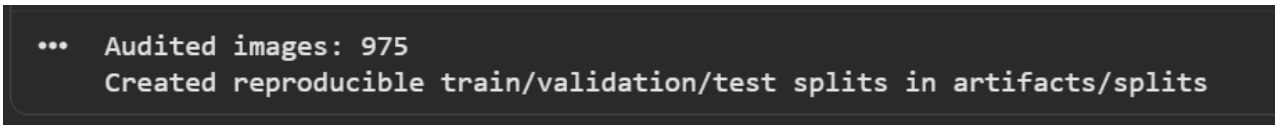
10_github_ci_failed.png *— Task 6.6 — CI lab evidence. This retained run shows the intentional failed workflow used to verify that the GitHub Actions checks respond to a failing test.*



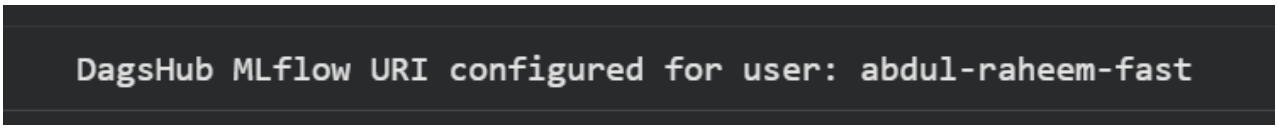
15_colab_environment_gpu.png *— Task 1.1 — Environment verification. The output records the runtime software and confirms that TensorFlow can detect the available GPU.*



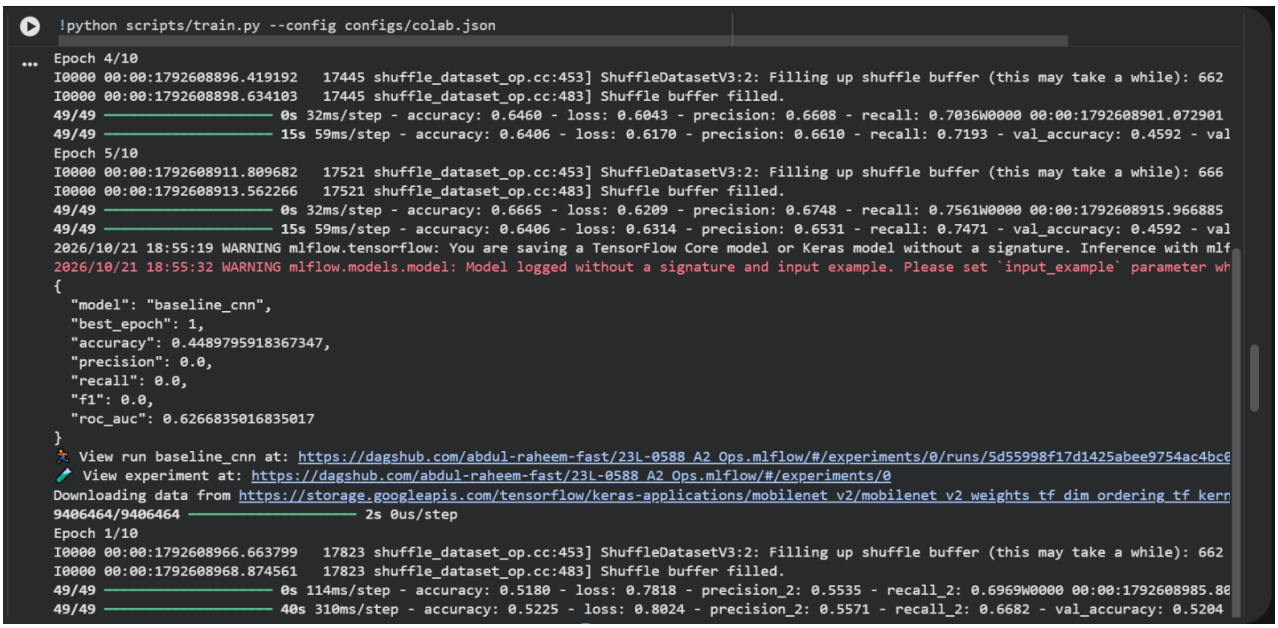
16_dataset_download_counts.png *— Task 1.2 — Dataset acquisition. The official dataset download is shown together with the source class counts used for the audit.*



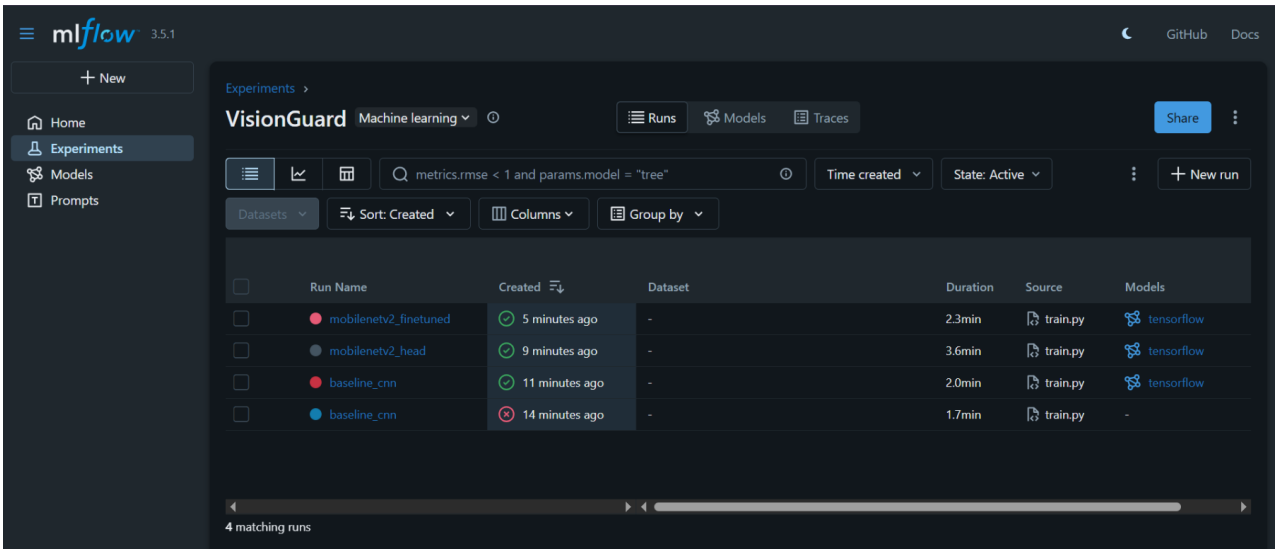
17_dataset_audit_splits.png *— Tasks 1.2 and 2.1 — Data quality and reproducibility. The audit result and seeded train, validation, and test split creation are shown together.*



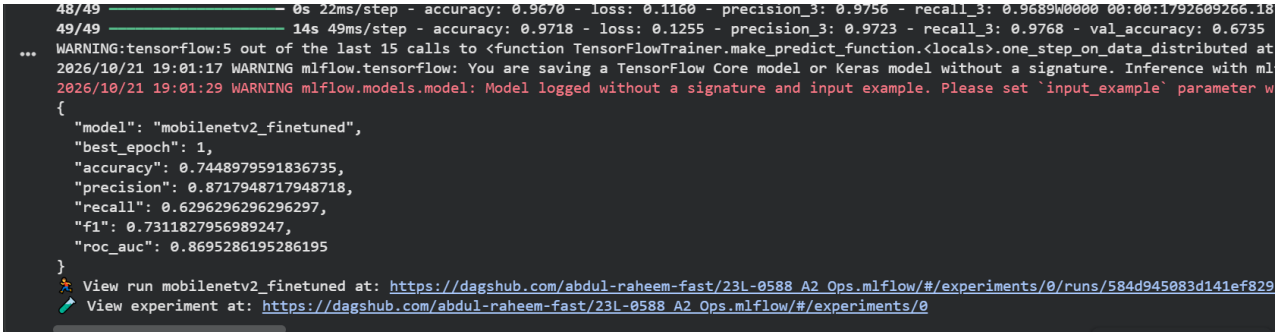
18_dagshub_mlflow_configuration.png *— Task 3.2 — DagsHub integration. The MLflow tracking URI is configured for the DagsHub project while the access token remains hidden.*



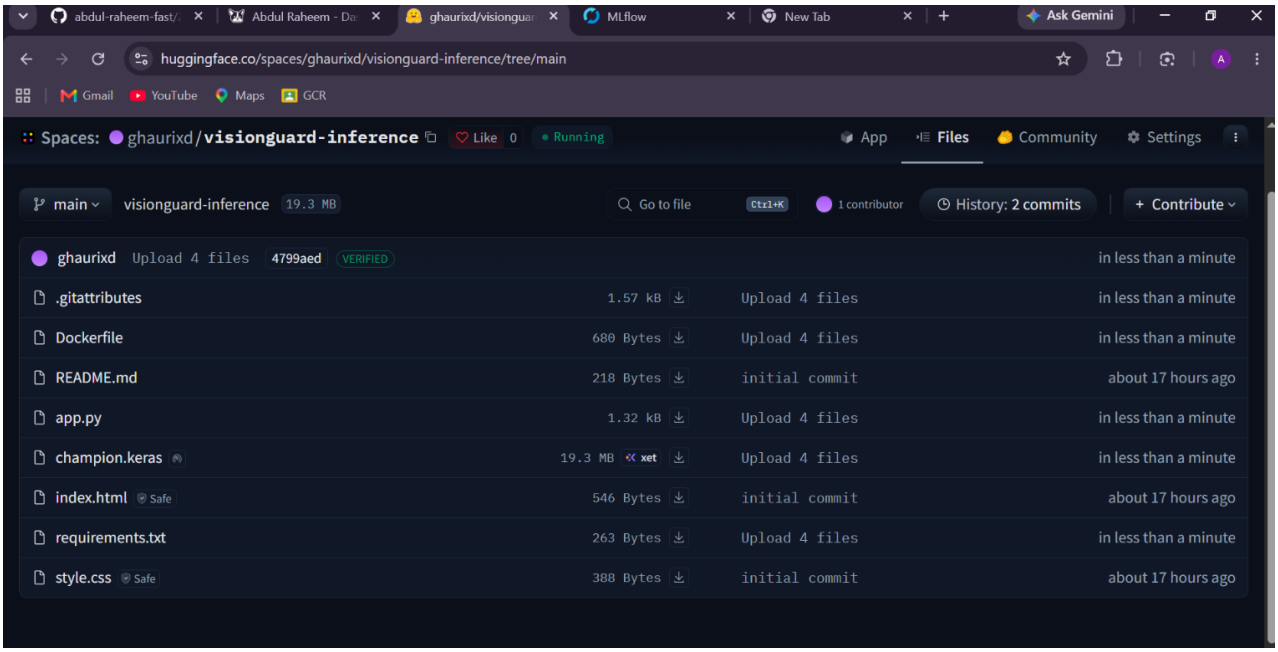
19_baseline_training_result.png *— Tasks 2.2, 2.4, 2.5, and 3.3 — Baseline CNN evidence. The training output shows the from-scratch model run, evaluation result, and tracked experiment context.*



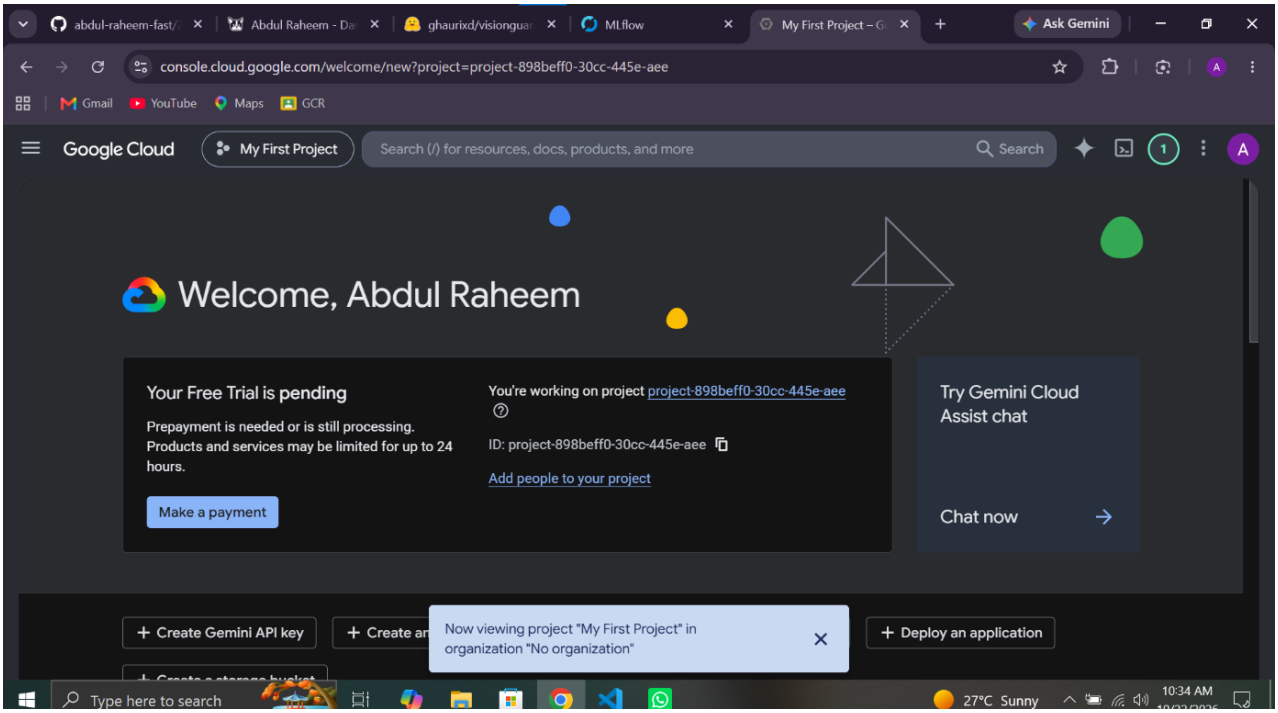
20_dagshub_final_runs.png *— Tasks 3.3 and 3.5 — Experiment comparison. The DagsHub MLflow project lists the completed baseline, transfer-learning, and fine-tuning runs.*



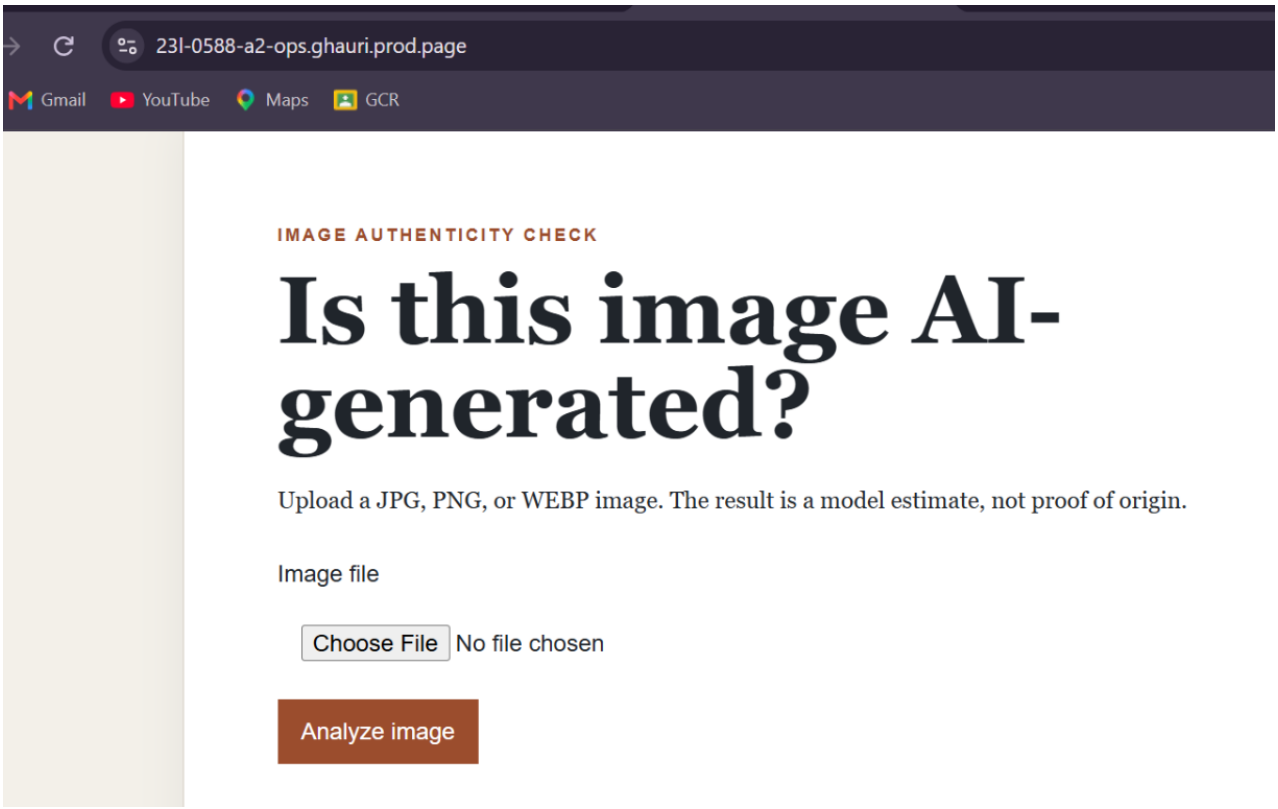
21_finetuned_training_result.png *— Tasks 2.3, 2.4, 2.5, and 3.3 — Champion training evidence. The MobileNetV2 fine-tuning output shows the selected approach and its recorded evaluation metrics.*



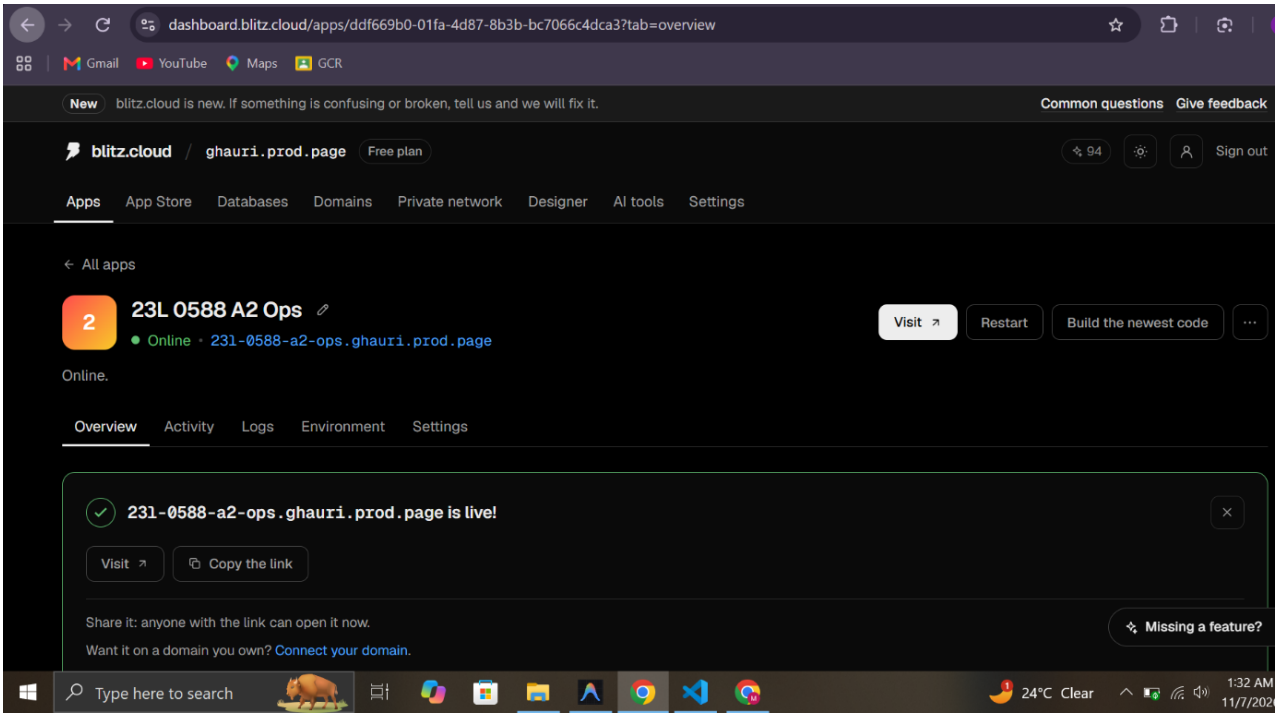
22_huggingface_space_files.png *— Task 5.3 — Hosting investigation. The Space file layout documents the inference-service deployment attempt and model placement diagnosis.*



23_google_cloud_trial_pending.png *— Task 5.3 — Hosting investigation. This evidence records the free-trial verification requirement encountered while evaluating Google Cloud.*



24_blitz_dashboard_online.png *— Tasks 7.1 and 7.2 — Cloud deployment. The Blitz dashboard shows the service online and provides the public deployment address.*



25_blitz_public_ui.png *— Tasks 7.2 and 8.1 — Hosted application verification. The public URL displays the deployed VisionGuard upload interface.*